

Complete SQL Practice Database with Questions & Answers

Prepared for SQL Practice and Interview Preparation

Create Database

```
CREATE DATABASE SQL_Practice;  
USE SQL_Practice;
```

Employees Table

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100),  
    department_id INT,  
    salary DECIMAL(10,2),  
    hire_date DATE  
);
```

Departments Table

```
CREATE TABLE Departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100)  
);
```

Customers Table

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(100)  
);
```

Products Table

```
CREATE TABLE Products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

Orders Table

```
CREATE TABLE Orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  total_amount DECIMAL(10,2),  
  region VARCHAR(50)  
);
```

Sales Table

```
CREATE TABLE Sales (  
  sale_id INT PRIMARY KEY,  
  product_id INT,  
  quantity INT,  
  sale_date DATE  
);
```

Returns Table

```
CREATE TABLE Returns (  
  return_id INT PRIMARY KEY,  
  customer_id INT,  
  return_date DATE  
);
```

Departments Data

```
INSERT INTO Departments VALUES  
(1, 'HR'),  
(2, 'IT'),  
(3, 'Finance'),  
(4, 'Sales');
```

Employees Data

```
INSERT INTO Employees VALUES
(101, 'Sanjit', 2, 90000, '2023-01-15'),
(102, 'Rahul', 2, 120000, '2022-05-20'),
(103, 'Priya', 1, 75000, '2023-03-12'),
(104, 'Amit', 3, 65000, '2021-07-01'),
(105, 'Neha', NULL, 85000, '2023-08-10'),
(106, 'Karan', 4, 95000, '2020-11-11'),
(107, 'Anjali', 4, 55000, '2022-02-18');
```

Customers Data

```
INSERT INTO Customers VALUES
(1, 'Rohit', 'Mumbai'),
(2, 'Simran', 'Delhi'),
(3, 'Arjun', 'Pune'),
(4, 'Sneha', 'Bangalore'),
(5, 'Vikas', 'Chennai');
```

Products Data

```
INSERT INTO Products VALUES
(201, 'Laptop', 70000),
(202, 'Phone', 30000),
(203, 'Tablet', 20000),
(204, 'Monitor', 15000),
(205, 'Keyboard', 2000);
```

Orders Data

```
INSERT INTO Orders VALUES
(1001, 1, '2023-01-10', 70000, 'West'),
(1002, 2, '2023-01-15', 30000, 'North'),
(1003, 1, '2023-02-12', 15000, 'West'),
(1004, 3, '2023-03-05', 20000, 'South'),
(1005, 4, '2023-03-18', 50000, 'East'),
(1006, 5, '2023-04-11', 30000, 'South'),
(1007, 2, '2023-05-20', 70000, 'North'),
(1008, 3, '2023-06-25', 15000, 'South');
```

SQL Practice Questions

1. Retrieve all employees.
2. Find employees with salary greater than 80000.
3. Show all customers from Mumbai.
4. Retrieve products with price greater than 20000.
5. Count total number of employees.
6. Find maximum salary.
7. Find minimum product price.
8. Show average employee salary.
9. Retrieve employees hired in 2023.
10. Find employees without department.
11. Find second highest salary.
12. Get top 3 highest paid employees.
13. Calculate total revenue per product.
14. Show count of orders per customer.
15. Find latest order placed by each customer.
16. Find products never sold.
17. Calculate average order value per customer.
18. Find customers with more than 1 order.
19. Show monthly sales revenue.
20. Find employees with salary between 50000 and 100000.
21. Rank employees by salary within department.
22. Find moving average of sales.
23. Calculate cumulative revenue by day.
24. Find customers with consecutive purchases.
25. Find churned customers.

26. Calculate revenue percentage contribution per product.
27. Retrieve last purchase for each customer.
28. Find longest gap between customer orders.
29. Calculate year-over-year growth.
30. Find top-performing department by average salary.

SQL Answers

1. Retrieve all employees

```
SELECT * FROM Employees;
```

2. Employees with salary > 80000

```
SELECT * FROM Employees WHERE salary > 80000;
```

3. Customers from Mumbai

```
SELECT * FROM Customers WHERE city = 'Mumbai';
```

4. Products price > 20000

```
SELECT * FROM Products WHERE price > 20000;
```

5. Count employees

```
SELECT COUNT(*) AS total_employees FROM Employees;
```

6. Maximum salary

```
SELECT MAX(salary) AS max_salary FROM Employees;
```

7. Minimum product price

```
SELECT MIN(price) AS min_price FROM Products;
```

8. Average employee salary

```
SELECT AVG(salary) AS avg_salary FROM Employees;
```

9. Employees hired in 2023

```
SELECT * FROM Employees WHERE YEAR(hire_date)=2023;
```

10. Employees without department

```
SELECT * FROM Employees WHERE department_id IS NULL;
```

11. Second highest salary

```
SELECT MAX(salary) FROM Employees WHERE salary < (SELECT MAX(salary) FROM Employees);
```

12. Top 3 highest paid employees

```
SELECT * FROM Employees ORDER BY salary DESC LIMIT 3;
```

13. Total revenue per product

```
SELECT s.product_id, SUM(s.quantity * p.price) AS revenue FROM Sales s JOIN Products p  
ON s.product_id = p.product_id GROUP BY s.product_id;
```

14. Count orders per customer

```
SELECT customer_id, COUNT(*) FROM Orders GROUP BY customer_id;
```

15. Latest order per customer

```
SELECT customer_id, MAX(order_date) FROM Orders GROUP BY customer_id;
```

16. Products never sold

```
SELECT p.* FROM Products p LEFT JOIN Sales s ON p.product_id = s.product_id WHERE  
s.product_id IS NULL;
```

17. Average order value per customer

```
SELECT customer_id, AVG(total_amount) FROM Orders GROUP BY customer_id;
```

18. Customers with more than 1 order

```
SELECT customer_id, COUNT(*) FROM Orders GROUP BY customer_id HAVING COUNT(*) >  
1;
```

19. Monthly sales revenue

```
SELECT MONTH(order_date), SUM(total_amount) FROM Orders GROUP BY  
MONTH(order_date);
```

20. Employees salary between 50000 and 100000

```
SELECT * FROM Employees WHERE salary BETWEEN 50000 AND 100000;
```

21. Rank employees by salary within department

```
SELECT employee_id, employee_name, department_id, salary, RANK() OVER (PARTITION  
BY department_id ORDER BY salary DESC) AS salary_rank FROM Employees;
```

22. Moving average of sales

```
SELECT order_date, total_amount, AVG(total_amount) OVER (ORDER BY order_date ROWS  
BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg FROM Orders;
```

23. Cumulative revenue by day

```
SELECT order_date, SUM(total_amount) OVER (ORDER BY order_date) AS  
cumulative_revenue FROM Orders;
```

24. Consecutive purchases

```
WITH cte AS (SELECT customer_id, order_date, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order_date FROM Orders) SELECT * FROM cte WHERE DATEDIFF(order_date, prev_order_date)=1;
```

25. Churned customers

```
SELECT customer_id, MAX(order_date) FROM Orders GROUP BY customer_id HAVING MAX(order_date) < DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

26. Revenue percentage contribution

```
WITH revenue_cte AS (SELECT s.product_id, SUM(s.quantity * p.price) AS revenue FROM Sales s JOIN Products p ON s.product_id = p.product_id GROUP BY s.product_id) SELECT * FROM revenue_cte;
```

27. Last purchase per customer

```
WITH ranked_orders AS (SELECT customer_id, order_id, order_date, total_amount, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) rn FROM Orders) SELECT * FROM ranked_orders WHERE rn=1;
```

28. Longest gap between orders

```
WITH cte AS (SELECT customer_id, order_date, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order_date FROM Orders) SELECT customer_id, MAX(DATEDIFF(order_date, prev_order_date)) FROM cte GROUP BY customer_id;
```

29. Year-over-year growth

```
WITH yearly_revenue AS (SELECT YEAR(order_date) AS year, SUM(total_amount) AS revenue FROM Orders GROUP BY YEAR(order_date)) SELECT year, revenue, revenue - LAG(revenue) OVER (ORDER BY year) AS yoy_growth FROM yearly_revenue;
```

30. Top-performing department

```
SELECT d.department_name, AVG(e.salary) AS avg_salary FROM Employees e JOIN Departments d ON e.department_id = d.department_id GROUP BY d.department_name ORDER BY avg_salary DESC;
```